

Project Report: AgriYield – Cloud-Based Rice Production Analysis

Student Name: Salman Ejaz Jathol

Data Structures - Semester Project

1. Introduction

For my final semester project, I chose to build **AgriYield**, a system designed to solve a real-world problem at my family's rice processing business (a rice sheller) in Pakistan. The facility processes thousands of kilograms of raw harvest daily, moving rice through dryers, shellers, and polishers.

Currently, our tracking is mostly manual. We buy raw harvest with varying moisture levels, store it, and process it, but we lack a digital way to track exactly how much "eatable" rice we get versus by-products like broken rice or husk. **AgriYield is my attempt to digitize this intake process using efficient Data Structures.**

2. The Problem

The main challenge in a rice sheller is **Efficiency Tracking**.

- **Moisture Management:** We receive batches with different moisture levels (e.g., 24% vs. 20%). High-moisture rice needs to be dried immediately or it spoils.
- **Supplier Tracking:** We buy from hundreds of different farmers. When a farmer arrives, we need to pull up their record instantly.
- **Data Volume:** We need to store thousands of batch records and be able to search them by date (e.g., "Show me all production from November").

3. Technical Implementation

Instead of using pre-made databases like SQL, I built the core storage engine from scratch in **C++** to demonstrate the data structures we learned in class.

A. The Database Engine (B-Trees)

I implemented a B-Tree (Degree 3) to act as the primary database.

- **Why:** B-Trees are optimized for disk storage and allow for efficient searching.
- **Usage:** Every time a new rice batch enters the factory, it gets an ID and is inserted into the B-Tree. This allows us to keep the data sorted and retrieve it quickly.

B. Supplier Directory (Hash Map)

To handle our list of farmers, I implemented a Hash Map with chaining.

- **Why:** We need instant access. When a truck arrives, I can't wait for a linear search.
- **Usage:** The system hashes the farmer's name to a specific index, giving us $O(1)$ (instant) access to their records.

C. Process Optimization (Priority Queue)

I implemented a Priority Queue to manage the drying schedule.

- **Why:** Not all rice is equal. A batch with 25% moisture is at higher risk than one with 20%.
- **Usage:** The system uses a Heap logic to flag the "Wettest" batch as the highest priority for the dryer, and the "Driest" batch as ready for processing.

4. Cloud Deployment (AWS)

A major requirement for this project was Cloud Computing. I deployed the AgriYield C++ engine on **Amazon Web Services (AWS)** using an **EC2 Ubuntu Instance**.

- **Connectivity:** The application runs on a cloud server, allowing it to be accessed remotely rather than just sitting on a local laptop.
- **Environment:** The code was compiled and executed in a Linux (Ubuntu) environment, proving that the system is **portable and scalable**.

5. Conclusion

AgriYield successfully **bridges the gap between theoretical code and industrial application**. By moving from paper logs to a B-Tree-based cloud system, we can now track our inventory digitally and prioritize our drying process automatically. This project **taught me how to** actually *apply* C++ to **solve business problems** rather than just writing algorithms in isolation.